Department of Computer Science

Banasthali Vidyapith

Software Requirement Specifications (SRS) and Software Design Specifications (SDS)

[Project Name- AI Driven Aircraft Maintenance Prediction System]

[Group ID- CSD009]

Guided By:

[Dr. Sneha Asopa]

Submitted By:

[Anshika Nagpal-2316008]

[Charu Solanki- 2316026]

[Ishita Jain- 2316043]

[Kanishka Jain- 2316049]

[Kaushiki Mishra- 2316050]

[Class- B.Tech (CS-AI) V Sem]

Chapter 1 SRS

Software Requirements Specification (SRS)

AI-Driven Aircraft Maintenance Prediction System

Contents

1 Introduction **2**

1.1 Purpose 2

1.2 Scope 2

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Overview 3

2 General Description **3**

2.1 Product Perspective 3

2.1.1 Product Functions 4

2.1.2 Hardware Interfaces 4

2.1.3 Software Interfaces 4

2.1.4 Communication Interfaces 5

2.2 User Characteristics 5

2.3 General Constraints 6

2.4 Technologies Used 6

3 Specific Requirements **7**

3.1 Functional Requirements 7

3.2 Non-Functional Requirements 8

3.2.1 Availability 8

3.2.2 Security 8

3.2.3 Reliability 8

3.2.4 Portability 8

3.2.5 Maintainability 8

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a detailed description of the requirements for the AI Driven Aircraft Maintenance Prediction System. Its purpose is to define the functional and non-functional requirements of the system, establishing a clear agreement on the product to be built between the development team and stakeholders. The intended audience for this document includes project managers, system designers, developers, and quality assurance teams who will use it as a foundational guide for system design, development, and testing.

1.2 Scope

This project, named AI-Driven Aircraft Maintenance Prediction System, will be a software-based prototype of an AI-driven predictive maintenance system for aircraft.

The system will:

- Analyze simulated or publicly available aircraft sensor and log datasets (e.g., NASA CMAPSS).
- Apply Machine Learning (ML) models to detect operational anomalies and predict component failures.
- Provide a dashboard interface for visualizing aircraft health status, failure predictions, and alerts.
- Recommend preventive maintenance actions based on model outputs.

The primary objective is to develop a proof-of-concept that demonstrates the benefits of applying AI/ML to aviation maintenance. The goal is to enhance operational safety, optimize resource utilization, reduce maintenance costs, and improve mission readiness for the Indian Air Force (IAF).

1.3 Definitions, Acronyms, and Abbreviations

- AI: Artificial Intelligence
- CMAPSS: Commercial Modular Aero-Propulsion System Simulation
- CSV: Comma-Separated Values
- DBSCAN: Density-Based Spatial Clustering of Applications with Noise

- IAF: Indian Air Force
- ML: Machine Learning
- NASA: National Aeronautics and Space Administration
- PCA: Principal Component Analysis
- RAM: Random Access Memory
- RUL: Remaining Useful Life
- SGD: Stochastic Gradient Descent
- SRS: Software Requirements Specification
- SVM: Support Vector Machine
- t-SNE: t-Distributed Stochastic Neighbor Embedding
- UI: User Interface

1.4 Overview

This SRS outlines: This SRS is organized into three main sections.

- Section 1 provides a high-level introduction, outlining the project's purpose, scope, and definitions.
- Section 2 offers a general description of the product, its perspective, user characteristics, and constraints, intended for all stakeholders to understand the system's context.
- Section 3 details the specific functional and non-functional requirements of the system, providing the technical specifications necessary for designers, developers, and testers to build and verify the product.

2 General Description

2.1 Product Perspective

The AI-Driven Aircraft Maintenance Prediction System is a self-contained, standalone software prototype designed to demonstrate the capabilities of predictive maintenance in the aviation sector. It serves as an analytical tool that ingests historical and simulated

operational data to produce actionable insights. While it is not part of a larger system, it is designed to simulate how such a product would integrate into the maintenance workflow of an aviation authority like the IAF.

The system operates as a "black box" that takes operational data as input and produces health assessments, failure predictions, and maintenance recommendations as output via a user-facing dashboard.

2.1.1 Product Functions

The major functions of the software are:

1. **Data Ingestion and Processing:**The user can upload aircraft sensor data files. The system will preprocess this data to prepare it for analysis.

2. **Failure Prediction:** Using trained AI/ML models, the system will analyze the data to predict the Remaining Useful Life (RUL) or time-to-failure for critical components.

3. **Anomaly Detection:** The system will identify unusual patterns or outliers in sensor readings that may indicate an impending fault.

4. **Fleet Management Dashboard:** An interactive dashboard will visualize the health status of aircraft, display predictions, highlight at-risk components, and provide maintenance alerts.

5. **Reporting:** Users can export analysis results and maintenance recommendations into PDF or CSV formats.

2.1.2 Hardware Interfaces

- No aircraft direct hardware interface requirements, as it does not control or interact with any external physical devices. It operates solely as a software application on a standard computer.

- The system is designed to run efficiently on a standard laptop or personal computer equipped with at least 8 GB of RAM and an Intel i5 (or equivalent) processor

2.1.3 Software Interfaces

1. Python Programming Language

    - Name: Python

- Version number: 3.10+
- Source: Open Source (Python Software Foundation)

2. Machine Learning Framework
   - Name: TensorFlow / PyTorch
   - Version number: Latest Stable Version
   - Source: Open Source (Google / Meta AI)
3. Frontend Web Framework
   - Name: Streamlit
   - Version number: Latest Stable Version
   - Source: Open Source
4. Data Visualization Library
   - Name: Matplotlib / Plotly
   - Version number: Latest Stable Version
   - Source: Open Source

2.1.4 Communication Interfaces

The system operates in a self-contained, offline environment by design. Its core functionality is executed entirely on the local machine, which means it has no network or internet dependencies. This standalone architecture is a deliberate security measure to ensure sensitive data is never transmitted.

2.2 User Characteristics

The intended users of this system include:

1. **Maintenance Engineers:** Technical personnel responsible for aircraft repair and upkeep. They will use the system to get specific maintenance recommendations.
2. **Fleet Managers:** Personnel overseeing the operational readiness of a fleet of aircraft. They will use the dashboard for high-level health monitoring and strategic planning.
3. **Data Analysts:** Personnel who may use the system's outputs to conduct deeper investigations into fleet-wide performance trends.

4. **Operational Staff:** Personnel at the Indian Air Force (IAF) and commercial airlines who monitor daily mission readiness and flight schedules. They will use the system's alerts and dashboard to make real-time decisions about aircraft availability and to prevent unexpected delays or grounding.

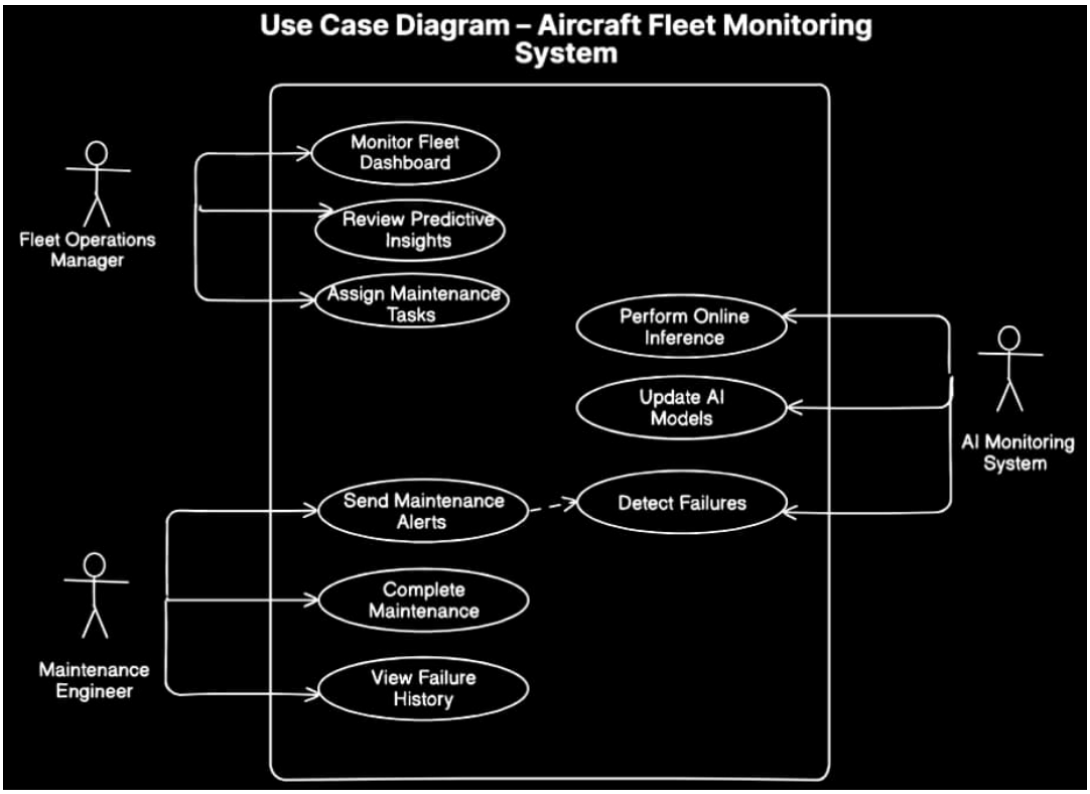


Figure 1: Use Case Diagram

2.3 General Constraints

1. **Data Security:** All data processing must be performed on a local machine. The system shall not transmit sensitive aircraft data over any network.

2. **Dataset Dependency:** The prototype will be developed and validated using publicly available datasets like NASA CMAPSS. Its performance on proprietary, real-world data is not guaranteed.

3. **Hardware Limitations:** The system must be capable of running efficiently on a standard workstation with the specified hardware (8GB RAM, i5 processor or better).

2.4 Technologies Used

- **Backend:** Python 3.10+

- **Machine Learning:** TensorFlow or PyTorch, Scikit-learn
- **Data Handling:** Pandas, NumPy
- **Algorithms:** Isolation Forest, Kalman Filter, Stochastic Gradient Descent (SGD), One-Class SVM, DBSCAN, K-Means, PCA, t-SNE.

3 Specific Requirements

3.1 Functional Requirements

1. **Data Upload:** The system shall allow users to upload aircraft sensor/log data in CSV format.
2. **Data Preprocessing:** The system shall preprocess and validate the input data for quality and format consistency.
3. **Model Training:** The system shall train and evaluate AI/ML models using the uploaded data.
4. **Failure Prediction:** The system shall predict the time-to-failure or fault risk for aircraft components.
5. **Results Display:** The system shall display prediction results on the dashboard, including a confidence score.
6. **Maintenance Recommendation:** The system shall recommend a preventive maintenance timeline based on the predictions.
7. **Report Export:** The system shall allow users to export reports (PDF/CSV) of the analysis and predictions.
8. **Anomaly Detection:** The system shall use algorithms like Isolation Forest to detect anomalies in streaming data.
9. **Online Learning:** The system shall support periodic retraining of models and use algorithms like Stochastic Gradient Descent for real-time classification.
10. **Data Visualization:** The system shall visualize high-dimensional health metrics using dimensionality reduction techniques like PCA and t-SNE.
11. **Clustering:** The system shall group aircraft by risk level or failure patterns using clustering algorithms like DBSCAN or K-Means.

12. **Alert System:** The system shall generate and prioritize high-severity alerts based on business rules.

3.2 Non-Functional Requirements

3.2.1 Availability

- The system shall be capable of processing a 100MB dataset (ingestion, preprocessing, and prediction) in under 10 seconds on the specified hardware.
- The dashboard UI shall load and respond to user interactions within 2 seconds.

3.2.2 Security

- All data processing, including model training and prediction, shall be performed exclusively on the user's local machine.
- The application shall not initiate any outbound network connections to transmit operational or user data.

3.2.3 Reliability

- The system's failure detection and predictive models shall achieve an accuracy of 85% or greater on the test dataset.
- The application shall handle data processing errors gracefully and provide informative error messages to the user without crashing.
- A backup of trained models and configuration settings shall be maintained to prevent loss of work in case of an unexpected shutdown.

3.2.4 Portability

- The system shall be deployable on common operating systems (Windows, macOS, Linux) that support Python 3.10+.

3.2.5 Maintainability

- The source code shall be well-documented and follow modular design principles to facilitate future updates and modifications.
- The system shall follow the documented code and project conventions to ensure consistency.

Chapter 2 SDS

Contents

1 Introduction **2**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
  - 1.2.1 Objectives of Scope . . . 3
  - 1.2.2 Terms, Definitions, Acronyms and Abbreviations . . . 3
- 1.3 Overview of Document . . . 5

2 System Architectural Design **6**
- 2.1 High Level Design Overview . . . 7
- 2.2 Detailed Description of Components . . . 8

3 Structure and Relationship **19**
- 3.1 Data Design . . . 19
- 3.2 Dataset Schema Description – CMAPSS Jet Engine Simulated Data . . . 20

4 User Interface Design **22**
- 4.1 Description of the User Interface . . . 22
- 4.2 Screen Images . . . 23
- 4.3 Objects and Actions . . . 23

5 TYPES OF TESTS (With Implementation) **25**
- 5.1 Unit Tests . . . 25
- 5.2 System Tests . . . 26
- 5.3 Integration and Regression Tests . . . 27
- 5.4 Stress Tests . . . 27
- 5.5 Acceptance Tests and Staging . . . 28

6 Appendices **28**
- 6.1 Dataset Description . . . 28
- 6.2 Algorithm Parameters . . . 29
- 6.3 Sample Sensor Log . . . 29
- 6.4 Testing Evidence . . . 29
- 6.5 Glossary . . . 30

7 References **30**

1 Introduction

Modern defense aircraft are equipped with advanced avionics, propulsion systems, and mission-critical subsystems that generate vast volumes of sensor and log data during each sortie. This data captures real-time insights into structural integrity, engine performance, flight dynamics, and onboard systems. However, current maintenance protocols often rely on fixed schedules or manual inspections, which can overlook early indicators of component degradation or system failure—posing risks to mission success and pilot safety. This Software Design Specification outlines a predictive maintenance system tailored for defense aviation. Leveraging AI and ML algorithms, the system analyzes multimodal aircraft telemetry to detect anomalies, forecast failures, and recommend targeted maintenance actions. An interactive dashboard provides role-based insights for ground crews, engineers, and command units, enabling proactive decision-making and minimizing unplanned downtime. By integrating intelligent diagnostics into the defense maintenance workflow, the system enhances fleet reliability, operational efficiency, and combat readiness.

1.1 Purpose

This Software Design Specification (SDS) outlines the design and development plan for a predictive maintenance system tailored to defense aircraft, created as part of a collaborative academic project by a team of five engineering students. The SDS serves as a foundational document to guide the implementation of the system, which leverages AI and Machine Learning to analyze aircraft sensor data, detect anomalies, and forecast potential failures. The primary purpose of this SDS is to:

- Clearly define the system's architecture, components, and data flow
- Ensure alignment among team members during development and testing
- Provide a structured reference for faculty supervisors, evaluators, and future contributors
- Demonstrate the team's technical approach, problem-solving strategy, and understanding of real-world aviation challenges

By documenting the system's design in detail, this SDS supports efficient collaboration, facilitates milestone tracking, and ensures that the final prototype meets both functional and academic expectations.

1.2 Scope

This project focuses on building a software-based predictive maintenance system for aircraft fleets operated by the Indian Air Force (IAF) and commercial airlines. The system will utilize AI and Machine Learning to analyze aircraft telemetry data in real time, enabling early detection of system anomalies and forecasting potential component failures.

1.2.1 Objectives of Scope

- Develop a prototype that ingests multi-modal aircraft data (engine, hydraulics, avionics, etc.)
- Implement AI-driven failure detection and online learning algorithms
- Create a role-based dashboard for ground crews, engineers, and command units
- Simulate real-world scenarios to validate system performance
- Data preprocessing from aircraft logs
- Design and training of ML models for anomaly detection and failure prediction
- Development of an interactive dashboard with alert and recommendation features
- Testing the system using simulated or publicly available aviation datasets

1.2.2 Terms, Definitions, Acronyms and Abbreviations

This section lists important terms, definitions, acronyms, and abbreviations used throughout this SDS. It ensures clarity and consistency for all stakeholders.

AI	Artificial Intelligence – Technology that enables machines to simulate human intelligence for tasks such as prediction, classification, and decision-making.
ML	Machine Learning – A subset of AI that uses algorithms and statistical models to learn patterns from data and make predictions.
IAF	Indian Air Force – Primary stakeholder and end-user of the predictive maintenance system in the project context.
APMS	Aircraft Predictive Maintenance System – The software system described in this SDS, designed to analyze sensor data and predict failures.
FDR	Flight Data Recorder – Device that records flight parameters such as speed, altitude, and control inputs for post-flight analysis.
Telemetry	Data transmitted from the aircraft's onboard systems to ground systems for monitoring and analysis.

Anomaly Detection	The process of identifying data points, events, or observations that deviate from expected normal behavior.
RUL	Remaining Useful Life – The estimated operational lifespan of a component before failure.
CI	Continuous Integration – Development practice where code changes are automatically tested and merged to ensure stability.
PyTest	Python-based testing framework used for unit testing machine learning and data processing modules.
JUnit	Java-based testing framework used for backend components and service logic.
SDS	Software Design Specification – This document that details system architecture, components, and implementation plan.
Dashboard	Graphical user interface that displays health metrics, alerts, and readiness status of aircraft fleet in real-time.
Feature Engineering	The process of selecting, transforming, and creating input features for ML models to improve performance.
Hyperparameter Tuning	Process of optimizing the control parameters of ML models (e.g., learning rate, depth of trees) for better accuracy.
Stress Test	Testing method to evaluate system behavior under maximum load conditions to ensure reliability and performance.
Integration Test	Testing technique used to verify that individual modules or components work correctly when combined.
Regression Test	Testing performed to ensure that newly added features do not break previously working functionality.
Alert	System-generated notification triggered when an anomaly or predicted failure is detected.
Mission Readiness	A metric indicating whether an aircraft is fully operational and capable of deployment.

1.3 Overview of Document

This Software Design Specification (SDS) document is organized into multiple sections to ensure clarity, maintainability, and ease of reference for developers, faculty, and stakeholders. The document follows a top-down approach, starting with a high-level description and progressively moving towards detailed technical specifications.

- **Section 1 – Introduction:** Defines the purpose, scope, objectives, and terminology used throughout the document.
- **Section 2 – System Architectural Design:** Provides a detailed description of the system's architecture, including conceptual components, class diagrams, activity diagrams, and sequence diagrams.
- **Section 3 – Structure and Relationship:** Describes the internal data design, dataset schema, data formats, and interrelationships between system modules.
- **Section 4 – User Interface Design:** Details the dashboard components, their purpose, mock-up screens, and user interactions.
- **Section 5 – Types of Tests:** Outlines the testing methodology including unit tests, system tests, integration tests, regression tests, stress tests, and acceptance tests.
- **Section 6 – Appendices:** Includes dataset descriptions, algorithm parameters, sample logs, testing evidence, and glossary for quick reference.
- **Section 7 – References:** Lists all external sources, research papers, and datasets referred to in the project.

2 System Architectural Design

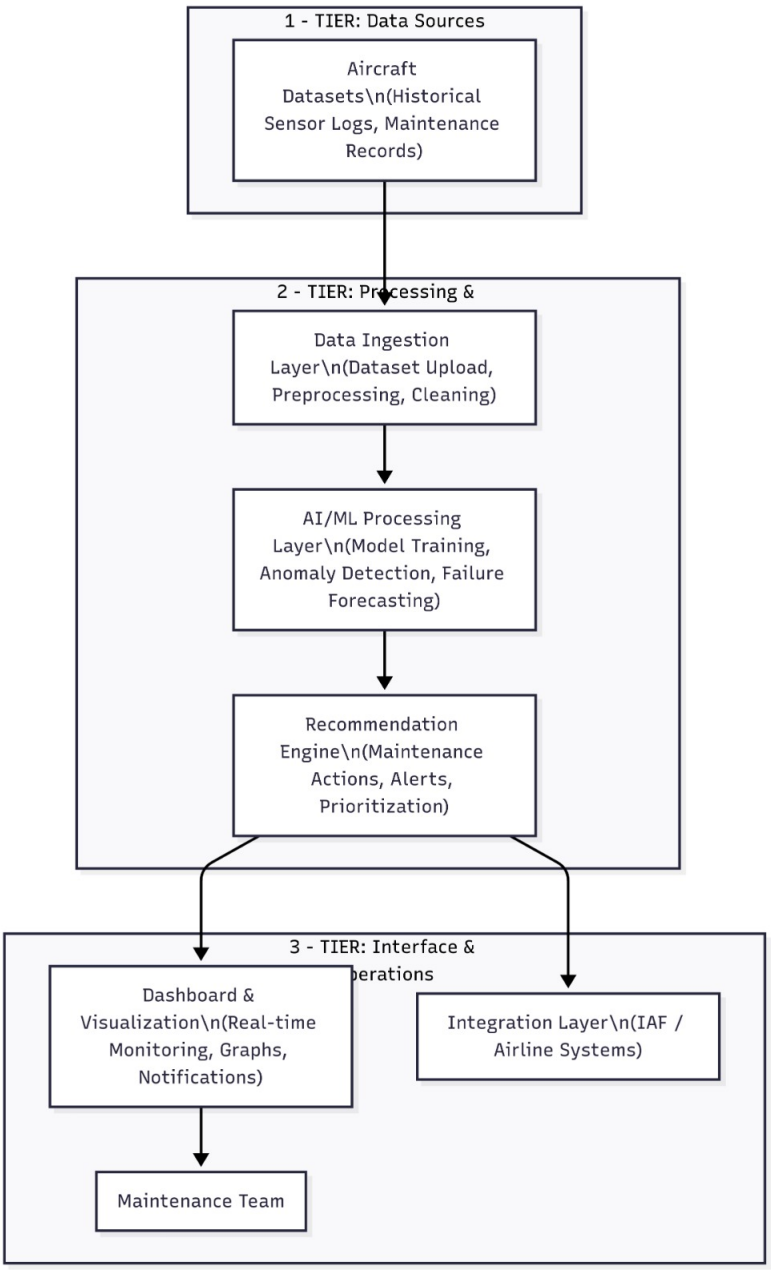


Figure 1: Architecture Diagram

2.1 High Level Design Overview

The predictive maintenance system is designed to analyze historical aircraft log data and forecast potential component failures using machine learning techniques. The architecture is modular, scalable, and optimized for clarity and maintainability. It supports offline model training, anomaly detection, and real-time visualization through a dashboard interface.

Conceptual Components

1. **Aircraft Log Data Source**
   - Static dataset containing historical telemetry, maintenance logs, and operational parameters.
2. **Data Ingestion Module**
   - Responsible for loading, parsing, and validating log data from structured files or repositories.
3. **Preprocessing and Feature Engineering**
   - Cleans and transforms raw data into structured formats.
   - Extracts relevant features such as temperature trends, pressure anomalies, and usage patterns.
4. **Anomaly Detection Engine**
   - Applies unsupervised learning or statistical techniques to identify deviations from normal behavior.
5. **Failure Prediction Module**
   - Uses time-series forecasting or classification models to predict potential component failures.
6. **Model Training and Evaluation Pipeline**
   - Supports offline training using historical data.
   - Includes validation, hyperparameter tuning, and performance metrics tracking.
7. **Interactive Dashboard**
   - Displays health scores, anomaly alerts, and predictive insights.

- Designed for maintenance personnel and supervisors to monitor aircraft status.

8. **User Access Layer**
    - Manages role-based access and permissions for different user types.

Pictorial Representation

The system architecture is represented vertically to reflect the flow of data and control across modules. The diagram illustrates the following key aspects:

- Left-to-right flow from *Aircraft Log Data Source* to *Dashboard*.
- Arrows indicating data movement and module interaction.
- Modular blocks representing each component in the system.
- External entities such as *Maintenance Technician* and *Aircraft System* positioned at the edges.

This architectural layout ensures that each module is independently testable, replaceable, and scalable. It supports both academic evaluation and future deployment in real-world scenarios by promoting modularity, clarity, and maintainability.

2.2 Detailed Description of Components

1. **Class Diagram**

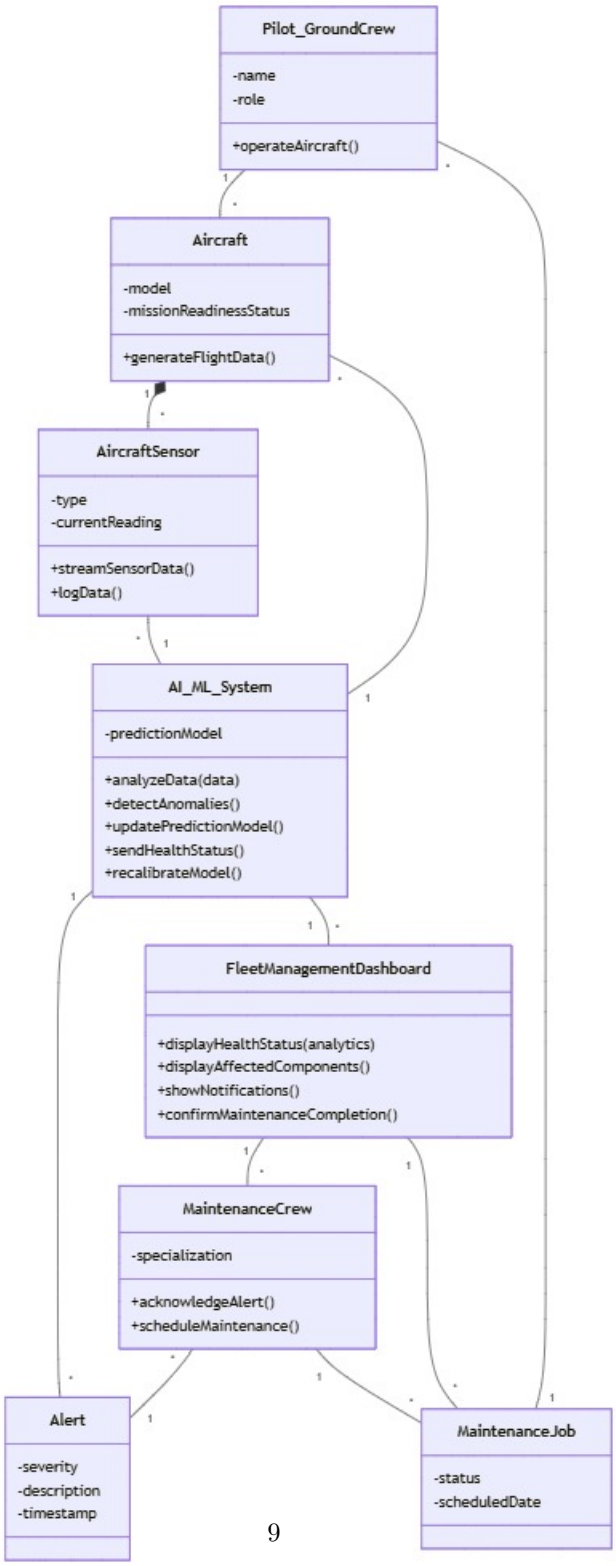


Figure 2: Class Diagram

The class diagram represents the object-oriented structure of the aircraft maintenance monitoring system. It defines the key entities, their attributes, operations, and interrelationships, providing a blueprint for implementation and modular design.

Purpose of the Diagram:

This diagram models the system's core components as classes, capturing their responsibilities and interactions. It supports maintainability, extensibility, and clear separation of concerns across operational, analytical, and user-facing modules.

Class Definitions and Responsibilities:

- **Pilot_GroundCrew** – Represents personnel responsible for operating and maintaining aircraft. Attributes: `name`, `role`. Operations: `operateAircraft()`, `maintainAircraft()`.
- **Aircraft** – Models individual aircraft units. Attributes: `model`, `serialNumber`, `status`. Operations: `operate()`, `getStatus()`.
- **AircraftSensor** – Represents onboard sensors collecting telemetry data. Attributes: `type`, `currentReading`. Operations: `retrieveSensorData()`, `calibrate()`.
- **AI_ML_System** – Encapsulates machine learning logic for predictive maintenance. Operations: `predictFailure()`, `analyzeData(sensorData)`, `trainModel(data)`, `validateModel(data)`, `updateModel(data)`.
- **FleetManagementDashboard** – Provides a user

interface for monitoring and managing aircraft status. Operations: `displayAircraftStatus()`, `updateAircraftComponents()`, `scheduleMaintenance()`, `notifyCrew()`.

- **MaintenanceCrew** – Represents engineers responsible for executing maintenance tasks. Attributes: `name`, `role`. Operations: `maintainAircraft()`, `scheduleMaintenance()`.
- **MaintenanceJob** – Models individual maintenance tasks. Attributes: `jobId`, `description`, `status`. Operations: `updateStatus()`, `addJobDetails()`.
- **Alert** – Represents system-generated notifications. Attributes: `severity`, `description`, `timestamp`.

Relationships and System Structure:

The diagram shows the following associations:

- `Pilot_GroundCrew` operates `Aircraft`.
- `Aircraft` contains multiple `AircraftSensor` instances.
- `AircraftSensor` sends data to `AI_ML_System`.
- `AI_ML_System` interacts with `FleetManagementDashboard` to display predictions.
- `FleetManagementDashboard` notifies `MaintenanceCrew`.
- `MaintenanceCrew` executes `MaintenanceJob`.
- `MaintenanceJob` may generate an `Alert`.

Justification and Implementation Relevance:

This class diagram supports modular development

and aligns with object-oriented principles. It enables:

- Clear mapping of system functionality to code structure.
- Reusability of components like `AircraftSensor` and `Alert`.
- Easy extension for new roles, sensors, or dashboard features.
- Logical encapsulation of ML workflows within `AI_ML_System`.

The diagram also informs database schema design, API structuring, and UI behavior by defining how data and control flow between entities.

2. **Activity Diagram**

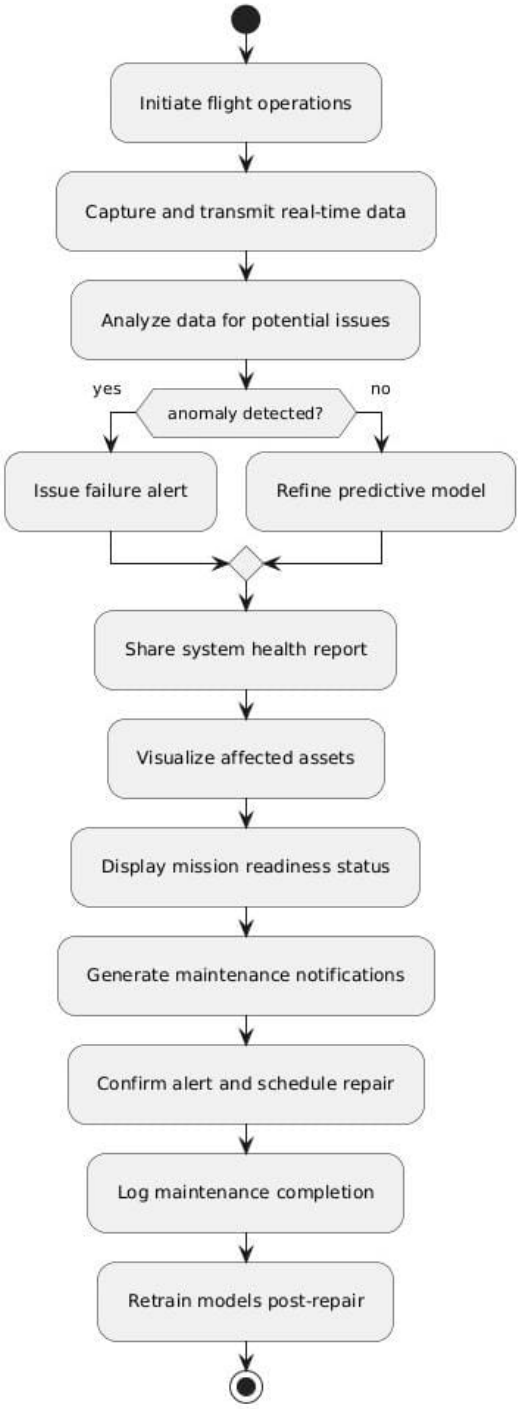


Figure 3: Activity Diagram

The activity diagram illustrates the operational workflow of the aircraft maintenance system, capturing the sequence of actions, decision points, and transitions from aircraft operation to maintenance scheduling and completion. It highlights how sensor data is processed, analyzed, and acted upon to ensure mission readiness and safety.

Purpose of the Diagram:

This diagram models the dynamic behavior of the system, showing how activities are triggered, how decisions are made based on data analysis, and how alerts and maintenance actions are executed. It supports understanding of system logic and helps guide implementation of control flow.

Process Description:

- The process begins with aircraft operation and continuous streaming of sensor and log data.
- Data is collected and stored in the data warehouse for analysis.
- The system analyzes incoming data to detect anomalies.
- A decision point checks whether an anomaly is present:
  - If an anomaly is detected, a failure alert is sent and the prediction model is updated.
  - If no anomaly is detected, a health status report is sent.

- Models are recalibrated periodically to improve accuracy.
- Critical alerts are triggered if necessary, and mission readiness is shared.
- Affected aircraft are displayed for review.
- Maintenance alerts are acknowledged, and maintenance is scheduled.
- Compliance is confirmed, and the process ends.

Transitions and Decision Logic:

The diagram includes conditional branches based on anomaly detection results. Transitions between activities are triggered by data availability, alert thresholds, and user acknowledgment. Loops are implied in model recalibration and continuous monitoring.

Justification:

This activity diagram provides a clear view of the system's operational logic and decision-making flow. It supports implementation by:

- Defining control flow for backend processing.
- Clarifying alert generation and response mechanisms.
- Informing UI behavior for dashboard updates and maintenance scheduling.
- Supporting modular development of each activity block.

The diagram ensures that the system is responsive, data-driven, and aligned with real-world aircraft maintenance workflows.

3. **Sequence Diagram**

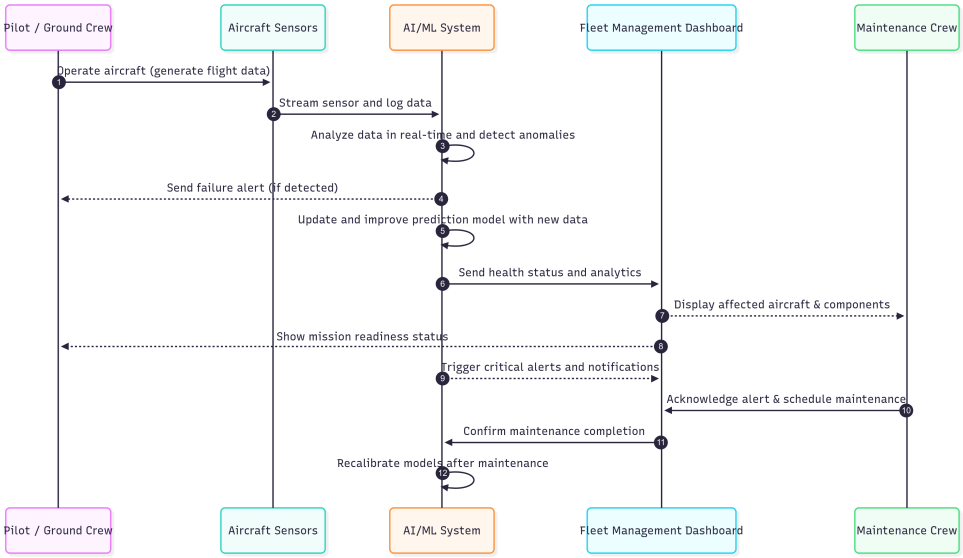


Figure 4: Sequence Diagram

Purpose:
This sequence diagram illustrates the flow of interactions among different entities in the Aircraft Predictive Maintenance System. It shows how flight data is generated, transmitted, processed by AI/ML models, and communicated to maintenance teams for proactive actions.

Actors / Participants:

- **Pilot / Ground Crew** – Operates the aircraft and generates flight data.

- **Aircraft Sensors** – Continuously monitor aircraft subsystems such as engine, hydraulics, avionics, and log flight data.
- **AI/ML System** – Processes real-time sensor data, detects anomalies, updates prediction models, and generates health analytics.
- **Fleet Management Dashboard** – Provides visualization, alerts, and status updates for airline operators and engineers.
- **Maintenance Crew** – Receives alerts, schedules maintenance, and ensures corrective actions.

Sequence of Events:

(a) Pilot/Ground Crew operates aircraft, generating flight data.

(b) Aircraft Sensors capture and stream sensor and log data.

(c) AI/ML System analyzes data in real-time to detect anomalies.

(d) If a failure is detected, the system sends a failure alert.

(e) AI/ML System updates and improves the prediction model with new data.

(f) AI/ML System sends health status and analytics to the Fleet Management Dashboard.

(g) Dashboard displays affected aircraft and components.

(h) AI/ML System provides mission readiness status.

(i) Critical alerts and notifications are triggered.

(j) Maintenance Crew acknowledges alerts and schedules maintenance.

(k) Confirmation of maintenance completion is sent back.

(l) AI/ML System recalibrates models based on post-maintenance data.

Key Features Represented:

- Real-time data acquisition and analysis from aircraft systems.
- AI-driven anomaly detection and predictive maintenance modeling.
- Bidirectional communication between crew, dashboard, and AI system.
- Automation of maintenance scheduling with minimal human delay.
- Continuous model improvement using feedback loops.

Design Considerations:

- System must handle high-throughput, low-latency sensor data.
- Must ensure fault tolerance in case of communication failure.
- Security of aircraft operational data must be maintained.
- Predictive models must adapt continuously for accuracy.

3 Structure and Relationship

This section outlines the interrelationships between system modules and layers. The structure chart and activity diagram illustrate how data and control flow across components. Each module is designed to be independently testable and scalable. The system also relies on structured datasets instead of a traditional database, which are described in detail below.

3.1 Data Design

Internal Data Structures:

- `DataFrames` – Used for loading and manipulating CMAPSS datasets.
- `NumPy arrays` – Used for feature vectors and model inputs.
- `Dictionaries` – Used for mapping engine IDs to metadata and predictions.

Global Structures:

- `config.yaml` – Stores model parameters, thresholds, and file paths.
- Shared repositories – Used for storing preprocessed data and trained models.

Temporary Structures:

- Cleaned datasets – Filtered and normalized sensor readings.
- Feature vectors – Engineered features for ML models (e.g., rolling averages, deltas).
- Intermediate outputs – Model predictions, anomaly scores, and RUL estimates.
- Alert logs – JSON-formatted alerts with severity, timestamp, and affected unit.

Data Formats Used:

- `CSV` – Primary format for CMAPSS raw and cleaned data.
- `JSON` – Used for alert payloads and dashboard communication.
- `Pickle/Joblib` – Used for saving trained models.

Validation and Transformation:

- Missing values are imputed or removed during preprocessing.
- Sensor readings are normalized using min-max scaling.
- Time-series data is segmented into fixed-length windows for model input.
- Feature engineering includes statistical summaries and trend indicators.

3.2 Dataset Schema Description – CMAPSS Jet Engine Simulated Data

Source:
The dataset is sourced from NASA's CMAPSS repository. It simulates jet engine degradation under various operating conditions and fault modes.

Dataset Variants:

- FD001 – One operating condition, one fault mode.
- FD002 – Six operating conditions, one fault mode.
- FD003 – One operating condition, two fault modes.
- FD004 – Six operating conditions, two fault modes.

Schema Overview:
Each row in the dataset represents one cycle of a specific engine. The fields include:

- `unit_number` (int) – Unique engine identifier.
- `time_in_cycles` (int) – Cycle count for the engine.
- `operational_setting_1` to `operational_setting_3` (float) – Environmental and control settings.

- `sensor_measurement_1` to `sensor_measurement_21` (float) – Sensor readings.

Sample Values:

- `unit_number`: 1, 2, 3, ...
- `time_in_cycles`: 1, 2, 3, ...
- `operational_setting_1`: 0.002, 0.003, ...
- `sensor_measurement_1`: 518.67, 518.67, ...

Units:

- Sensor values are unitless or normalized.
- Time is measured in operational cycles.

Purpose of Each Dataset:

- Training ML models for Remaining Useful Life (RUL) prediction.
- Real-time inference for anomaly detection.
- Alert generation for maintenance scheduling.
- Dashboard visualization of engine health.

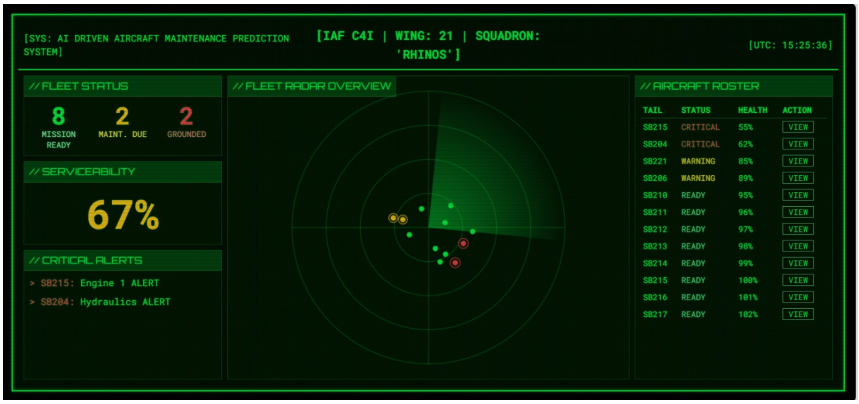
Integrity Constraints:

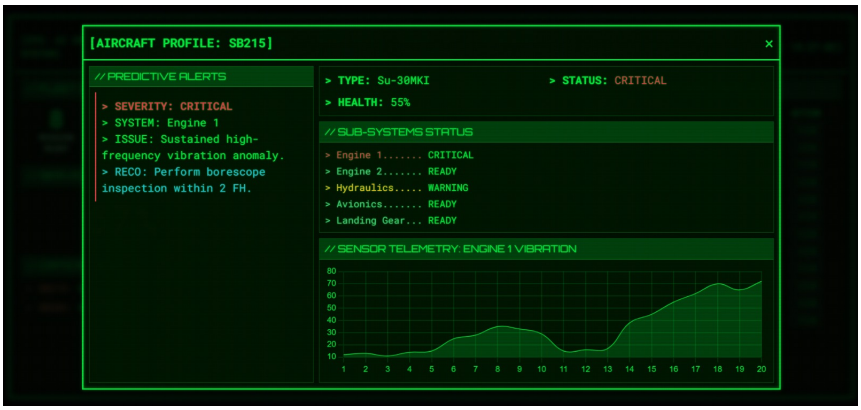
- `unit_number` must be unique per trajectory.
- `time_in_cycles` must be sequential and non-negative.
- Sensor values must fall within expected operational ranges.
- No null values are allowed in model input features.

File Format and Structure:

- `train_FDXXX.txt` – Training data with full degradation trajectories.
- `test_FDXXX.txt` – Test data ending before failure.
- `RUL_FDXXX.txt` – Ground truth RUL values for test data.

4 User Interface Design





4.1 Description of the User Interface

The Aircraft Predictive Maintenance System features a mission-critical, real-time dashboard designed for pilots, fleet operators, and maintenance crews. The interface follows a high-contrast, military-inspired HUD (Head-Up Display) style to maximize clarity in low-light environments and emphasize critical alerts.

The UI consists of:

- **Predictive Alerts Panel** – Displays severity, system affected, issue details, and recommended corrective actions.
- **Subsystems Status Panel** – Monitors engines, hydraulics, avionics, landing gear, and provides readiness state (READY,

WARNING, CRITICAL).

- **Sensor Telemetry Graphs** – Plots live sensor data (e.g., vibrations, temperature, hydraulic pressure) to visualize anomalies.
- **Fleet Overview Dashboard** – Shows mission readiness, serviceability percentage, grounded aircraft, and critical alerts.
- **Radar-style Visualization** – Displays aircraft health on a radar map with color-coded status (green = ready, yellow = warning, red = critical).
- **Aircraft Roster Panel** – Lists all aircraft with tail number, health percentage, and quick "VIEW" action for detailed inspection.

4.2 Screen Images

The following images represent the prototype UI from the end-user perspective:

1. **Aircraft Health Alert Screen** Displays predictive alert for *Engine 1* anomaly, subsystem status with color-coded severity, and real-time telemetry graph.
2. **Fleet Management Dashboard** Shows mission readiness statistics (ready, due for maintenance, grounded), fleet serviceability percentage, radar visualization, and aircraft roster.

4.3 Objects and Actions

Dashboard Components and Their Purpose

- **Predictive Alerts Panel**

  Displays anomaly alerts with severity, issue, and recommendations.

  Data Type: Text, Status Flags

  Properties: Severity levels – `CRITICAL`, `WARNING`, `READY`

  Response / Action: Alerts crew, highlights affected subsystem, and shows recommended corrective action.

- **Subsystems Status Panel**

  Shows health of engines, hydraulics, avionics, and landing gear.

  Data Type: Categorical Status

  Properties: Status – `READY`, `WARNING`, `CRITICAL`

  Response / Action: Updates in real time, with critical components highlighted in red.

- **Sensor Telemetry Graph**

  Plots live sensor data trends (e.g., engine vibrations).

  Data Type: Line Graph (numeric)

  Properties: X-axis = Time, Y-axis = Sensor Values

  Response / Action: Updates dynamically with incoming data.

- **Fleet Status Summary**

  Displays fleet-wide readiness (mission-ready, maintenance due, grounded).

  Data Type: Numeric / Counts

  Properties: Color-coded integer values

  Response / Action: Updates automatically and informs the operator of readiness.

- **Serviceability Meter**

  Shows percentage of operational fleet health.

  Data Type: Numeric (percentage)

  Properties: Value range: `0--100%` (Green ¿ 80%, Yellow 50–80%, Red ¡ 50%)

  Response / Action: Adjusts dynamically based on fleet condition.

- **Radar Visualization**

  Displays aircraft on a radar map with status indicators.

  Data Type: Graphical Objects

Properties: Aircraft dots – Green (Ready), Yellow (Warning), Red (Critical)

Response / Action: Updates dynamically; clicking a dot opens detailed status.

- **Aircraft Roster Table**

  Lists aircraft with Tail ID, Status, Health %, and actions.

  Data Type: Tabular Data

  Properties: Columns – Tail, Status, Health, Action

  Response / Action: `VIEW` button loads detailed aircraft profile.

- **VIEW Button (per aircraft)**

  Provides quick access to individual aircraft status and predictive alerts.

  Data Type: Clickable Button

  Properties: Linked to aircraft profile

  Response / Action: Loads aircraft-specific alert screen and telemetry.

5 TYPES OF TESTS (With Implementation)

Testing is a crucial phase in ensuring the reliability, safety, and effectiveness of the Aircraft Predictive Maintenance System (APMS). Since the system handles critical real-time sensor data, predictive modeling, and maintenance recommendations, multiple layers of testing are performed to validate correctness, performance, and safety compliance. The following testing strategies will be implemented:

5.1 Unit Tests

Unit testing will focus on validating individual components and modules of the system in isolation.

Scope:

- Sensor data preprocessing functions (e.g., data cleaning, normalization).

- Machine Learning model functions (anomaly detection, prediction).
- Dashboard UI components (alerts panel, fleet visualization).
- Utility functions (logging, data parsing, API endpoints).

Implementation:

- Developers will implement unit tests using *JUnit (Java components)* and *PyTest (Python-based AI/ML models)*.
- Mock datasets will be generated to simulate sensor inputs and edge cases.
- Each developer will run unit tests locally, and results will also be verified on a common development server.
- Continuous Integration (CI) pipelines will run all unit tests automatically on each commit.

5.2 System Tests

System testing validates end-to-end functionality by integrating multiple modules and executing realistic use-case scenarios.

Scope:

- Data flow from aircraft sensors → ingestion layer → AI/ML engine → dashboard → maintenance alerts.
- Failure forecast and alert generation in near real-time.
- Correct synchronization between dashboards and recommendation engine.

Implementation:

- A *test server* mirroring the target environment's OS will be used.
- Use-case scenarios (e.g., engine overheating anomaly, hydraulic pressure drop) will be simulated.
- End-to-end workflows such as anomaly detection → alert → crew acknowledgment → model update will be tested.

5.3 Integration and Regression Tests

Integration testing ensures that modules interact correctly, while regression testing verifies that new changes do not break existing features.

Scope:

- AI/ML anomaly detection working with real-time ingestion APIs.
- Alerts and notifications integrating with dashboards and crew communication channels.
- Dataset repository interactions.

Implementation:

- A *controlled integration environment* (with restricted access) will be maintained.
- Test datasets will be uploaded by a designated *Test Coordinator*.
- Regression suites will be automated using *Selenium (UI)*, *JUnit/PyTest (backend)*, and *Postman (API testing)*.

5.4 Stress Tests

Stress testing will validate system stability and performance under extreme load conditions.

Scope:

- High-volume sensor data streaming (e.g., 10x normal load).
- Large-scale fleet monitoring (hundreds of aircraft simultaneously).
- AI/ML inference speed under heavy load.

Implementation:

- Stress tests will be performed in an environment identical to the target hosting environment.
- Tools like *Apache JMeter / Locust* will simulate thousands of concurrent requests.

- Particular attention will be given to database performance and potential *deadlocks*.
- Tests will validate latency thresholds (alerts must be generated within <2 seconds).

5.5 Acceptance Tests and Staging

Acceptance testing ensures the system meets stakeholder requirements and is ready for production deployment.

Scope:

- Verification of all user stories: predictive alerts, health analytics, fleet readiness dashboard, maintenance recommendations.
- Validation by *airline operators, engineers, and maintenance crews*.
- Usability of dashboards (clarity, real-time updates, critical alert visibility).

Implementation:

- Acceptance tests will be performed in the *staging environment*, identical to production.
- Stakeholders will validate outputs using *real anonymized flight datasets*.
- After acceptance, the system will be promoted to production, and post-maintenance recalibration of AI/ML models will be verified.

6 Appendices

6.1 Dataset Description

The predictive maintenance system uses historical and real-time datasets from aircraft operations.

- **Engine Sensor Data** – Temperature, vibration, fuel flow, RPM readings.
- **Hydraulic Sensor Data** – Pressure, fluid levels, actuator signals.

- **Avionics Logs** – Navigation data, communication signals, autopilot status.
- **Flight Data Recorder Logs** – Altitude, speed, heading, control inputs.
- **Maintenance Records** – Historical records of component replacements and repairs.

These datasets are used for training anomaly detection models and validating prediction accuracy.

6.2 Algorithm Parameters

Details of machine learning algorithms and their hyperparameters:

- **Random Forest** – 200 trees, maximum depth = 15, min samples split = 5.
- **LSTM Neural Network** – 3 hidden layers, hidden size = 128, dropout = 0.3, learning rate = 0.001.
- **Anomaly Detection (Isolation Forest)** – contamination = 0.01, n_estimators = 150.

6.3 Sample Sensor Log

```
Timestamp: 2025-09-10 12:45:01
Engine_Temperature: 720C
Hydraulic_Pressure: 1800 psi
Avionics_Status: OK
Altitude: 32,000 ft
Speed: 820 km/h
...
```

6.4 Testing Evidence

- Unit testing conducted on anomaly detection functions using simulated datasets.
- System testing validated dashboard integration with live data streams.
- Stress testing identified database bottlenecks under 10,000 sensor records/second.

6.5 Glossary

AI Artificial Intelligence

ML Machine Learning

IAF Indian Air Force

FDR Flight Data Recorder

Anomaly Detection Identifying deviations from normal system behavior

7 References

References

[1] Kamal Haider, Jeffrey Tweedale, Lakhmi C Jain, and Pierre JM Urlings. Intelligent decision support feedback using multi-agent system in a defence maintenance environment. *International Journal of Intelligent Information and Database Systems*, 1(3-4):311–324, 2007.

[2] VRS Raju, OP Gandhi, and SG Deshmukh. Maintenance, repair, and overhaul performance indicators for military aircraft. *Defence Science Journal*, 62(2):83–89, 2012.

[3] Michael J Scott, Wim JC Verhagen, Marie T Bieber, and Pier Marzocca. A systematic literature review of predictive maintenance for defence fixed-wing aircraft sustainment and operations. *Sensors*, 22(18):7070, 2022.

[4] Sanjay Kumar Shukla, Satish Kumar, P Selvaraj, and V Subba Rao. Maintenance management system for a defence aircraft development programme. *International Journal of Product Lifecycle Management*, 8(1):65–79, 2015.